Типы `--payload` в zapret2

Параметр `--payload` определяет **тип содержимого пакета** (payload type), к которому будут применяться последующие LUA-функции desync.

📋 Полный список типов payload:

🔷 Специальные типы

1. `all` - все пакеты (любой payload)
2. `unknown` - неопознанный тип данных
3. `empty` - пустой пакет (без данных)
4. `known` - любой известный тип (все кроме `unknown` и `empty`)

🔷 HTTP

5. `http_req` - HTTP запрос (GET, POST и т.д.)
6. `http_reply` - HTTP ответ сервера

🔷 TLS/SSL

7. `tls_client_hello` - TLS Client Hello (начало TLS handshake от клиента)
8. `tls_server_hello` - TLS Server Hello (ответ сервера)

🔷 QUIC

9. `quic_initial` - QUIC Initial пакет

🔷 WireGuard VPN

10. `wireguard_initiation` - WireGuard инициация соединения
11. `wireguard_response` - WireGuard ответ
12. `wireguard_cookie` - WireGuard cookie пакет
13. `wireguard_keepalive` - WireGuard keepalive
14. `wireguard_data` - WireGuard данные

🔷 P2P и мессенджеры

15. `dht` - DHT (Distributed Hash Table, используется в BitTorrent)
16. `discord_ip_discovery` - Discord IP discovery пакет
17. `stun` - STUN протокол (используется для NAT traversal)

🔷 XMPP (Jabber)

18. `xmpp_stream` - XMPP stream
19. `xmpp_starttls` - XMPP STARTTLS команда
20. `xmpp_proceed` - XMPP proceed ответ
21. `xmpp_features` - XMPP features

🔷 DNS

22. `dns_query` - DNS запрос
23. `dns_response` - DNS ответ

🔷 Telegram

24. `mtproto_initial` - MTProto initial пакет (протокол Telegram)

💡 Синтаксис использования:

```
# Один тип
--payload=tls_client_hello

# Несколько типов (через запятую)
--payload=http_req,http_reply

# Все известные типы
--payload=known

# Все пакеты
--payload=all

# Отрицание (все КРОМЕ указанных)
--payload=~empty
```

📝 Важные особенности:

1. **По умолчанию**: если `--payload` не указан, используется `known` (только известные непустые типы)
2. **Отрицание**: префикс `~` означает инверсию
   - `--payload=~empty` - все кроме пустых пакетов
   - `--payload=~unknown` - все кроме неопознанных
3. **Множественные фильтры**: можно использовать несколько раз в одном профиле
   ```
   --payload=tls_client_hello --lua-desync=fake \
   --payload=http_req --lua-desync=split
   ```
4. **Связь с протоколом**:
   - Протокол соединения (`--filter-l7`) определяется для всего соединения
   - Payload type определяется для каждого пакета отдельно
   - Пример: протокол `tls` , но payload может быть `tls_client_hello` или `tls_server_hello`

📖 Примеры использования:

```
# Только для TLS Client Hello
--filter-l7=tls --payload=tls_client_hello --lua-desync=fake

# Для HTTP запросов и ответов
--payload=http_req,http_reply --lua-desync=multisplit

# Для всех известных типов (по умолчанию)
--payload=known --lua-desync=fake

# Для всех пакетов включая пустые
--payload=all --lua-desync=send

# Исключить пустые пакеты
--payload=~empty --lua-desync=fake

# Комбинация: разные стратегии для разных payload
--payload=tls_client_hello --lua-desync=fake:ip_ttl=1 \
--payload=http_req --lua-desync=split:pos=method+2
```

Типы payload позволяют **точно таргетировать** desync-стратегии на конкретные типы данных в пакетах, что делает обход DPI более эффективным и избирательным!

Фильтр payload по умолчанию

```
function payload_match_filter(l7payload, l7payload_filter, def)
    local argpl = l7payload_filter or def or "known"
    local neg = string.sub(argpl,1,1)=="~"
    local pl = neg and string.sub(argpl,2) or argpl
    return neg ~= (in_list(pl, "all") or in_list(pl, l7payload) or in_list(pl, "known")
and l7payload~="unknown" and l7payload~="empty")
end
```

По умолчанию фильтр = "known", что означает:

- ✅ Пропускает любой известный payload (`mtproto_initial` , `tls_client_hello` , и т.д.)
- ❌ **НЕ пропускает** `unknown`
- ❌ **НЕ пропускает** `empty`

Для MTProto

Пакет	l7payload	Пройдёт "known"?
Первый пакет с данными	`mtproto_initial`	✅ Да
Последующие пакеты	`unknown`	❌ **Нет**

Нужно ли указывать `payload=unknown` ?

Зависит от цели:

Если нужен только initial (обычный случай):

```
# Достаточно - unknown пакеты игнорируются автоматически
nfqws2 --filter-l7=mtproto \
  --payload=mtproto_initial \
  --lua-desync=fake:blob=0x00000000
```

Если нужны ВСЕ пакеты MTProto:

```
# Вариант 1: явно указать unknown
nfqws2 --filter-l7=mtproto \
  --payload=mtproto_initial --lua-desync=fake:blob=... \
  --payload=unknown --lua-desync=send:ip_ttl=3

# Вариант 2: использовать all
nfqws2 --filter-l7=mtproto \
  --payload=all \
  --lua-desync=fake:blob=...
```

Практический совет

Для обхода блокировки Telegram обычно **достаточно** `mtproto_initial`, потому что:

1. DPI анализирует только начало соединения (сигнатуру MTProto)
2. После initial все данные зашифрованы - DPI их не парсит
3. Если initial прошёл - соединение установлено

```
# Рекомендуемый вариант
nfqws2 --filter-l7=mtproto \
  --payload=mtproto_initial \
  --lua-desync=fake:blob=0x00000000:repeats=3
```

Ответ: Нет, обычно НЕ нужно указывать `payload=unknown` для MTProto — достаточно обработать только `mtproto_initial`.